Lab 11

Objective: Learn about `structs`, `unions`, and bit fields
This lab builds on lectures 8, 9, 10, and 11

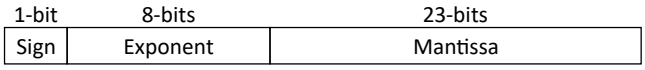
Exercise 1: Should-a done the homework!
A shallow copy is a byte-for-byte copy of an object (a `struct` or `union`). A deep copy is a replication of an object. What do we mean by this? If we have a queue (such as the one discussed in the lecture), and we want a second copy of that queue that works independently of the first queue, then we need a new queue that contains the same information as the first. Only then will the two queues work independently.

The homework question was to write a routine to deep copy a queue. If you already did the homework, then great – you've done this exercise. If not, then write the necessary routines to deep copy a queue. You must do this without destroying the original queue. The routine *prototype* should be:

```
queue *queue_deep_copy(queue *this) ;
```

Exercise 2: Really, you should-a done the homework.
Is the computer you're using big-endian or little-endian? Use a `union` to find out. If you don't know these terms then read the Wikipedia article on *Endianness*. Of particular amusement is the *Etymology* of these terms (read the *origin* section of the Wikipedia article).

Exercise 3: What didn't you get about doing the homework?
Floating point numbers (`float` in C) are typically stored in 32-bits using the IEEE 754 single precision format. That format is 1 bit for the *sign* (positive or negative), 8 bits for the *exponent*, the 23 bits for the *mantissa*. It looks like this:



Value = $(-1)^S$ X 1.M X 2^{E-127}

We can use a bit field to represent this. Go ahead and write the declaration for the bit field. It'll look something like this:

```
struct ieee
   {
   unsigned int mantissa:23;
   unsigned int exponent:8;
   unsigned int sign:1;
   };
```

Why does this declaration put the mantissa first?

Now you can write a `union` that contains 1 `float` and 1 `struct ieee`.

If we write into the `float` in the `union` then we can read the individual parts from the `struct ieee` in the union. But, in order to make sure you got it right you're going to write a routine that, given an integer and a number of bits, will write out the bit pattern of that integer in up-to that number of bits.

From these pieces, write a program that given a `float` (use "`%f`" in `scanf()`) will print out 1 bit for the sign, 8 bits for the exponent, then 23 bits for the mantissa. Use `float`, do not use `double`. Make sure your program is correct. Here's an example:

85.125 should result in `0 10000101 01010100100000000000000`